

Airflow CI/CD Pipeline Interview Notes

What I Built

I built a production-style Airflow pipeline on Google Cloud with GitHub as the source of truth and Cloud Build as the deployment mechanism. The final runtime is Cloud Composer (managed Airflow).

The flow is:

1. Developer changes code in GitHub
2. GitHub push triggers Cloud Build
3. Cloud Build syncs DAGs, scripts, and SQL into the Composer bucket
4. Composer detects the updated DAG
5. The monday DAG runs in order:
 - load data into BigQuery
 - read the BigQuery table and write to Google Sheets

Main Goal

The goal was to make a learning pipeline that behaves like a real data engineering workflow:

- version-controlled code
- automated deployment
- managed orchestration
- secure authentication
- clear separation between SQL, Python logic, and Airflow orchestration

Google Cloud Services Used

- **Cloud Composer:** managed Airflow runtime
- **Cloud Build:** CI/CD deployment from GitHub
- **BigQuery:** warehouse used for the sample table
- **Google Sheets API:** writes final output to a spreadsheet
- **Google Drive API:** required by gspread when opening/creating the sheet
- **Secret Manager:** optional credential source
- **Cloud Logging:** task logs from Composer

Repository Structure

Important folders in `learning_airflow`:

- dags/
- scripts/
- sql/
- requirements.txt
- cloudbuild.yaml

What Each File Does

1. dags/monday.py

This is the main business DAG.

Purpose: - define the Airflow workflow - run the SQL load task first - run the Google Sheets load task second

Behavior: - schedule: every Monday at 07:00 - task order: - run_sql_first - load_sheet_second

Why it matters: - this is the DAG interviewers will care about as the orchestration layer

2. dags/util/common_utils.py

This contains shared Airflow helper code.

Purpose: - standard default DAG arguments - timezone handling - reusable subprocess runner for Python scripts

Key function: - run_python_script(script_name)

What it does: - builds the script path inside Composer runtime - injects PYTHONPATH - runs the script with the current Python executable - raises a clear error if the script fails

Why it matters: - avoids duplicating script-launch logic in every DAG - keeps DAG code clean

3. scripts/load_sample_data_to_bq.py

This script loads sample data into BigQuery.

Purpose: - read SQL from sql/load_sample_bq_table.sql - replace placeholders for project/dataset/table - ensure dataset exists - execute the CREATE OR REPLACE TABLE query

Important parts: - detects credentials through google_auth_utils.py - creates the dataset if missing - runs SQL through the BigQuery client

Why it matters: - this is the first step in the pipeline - it refreshes the source table before the sheet export

4. `scripts/load_bq_to_gsheet.py`

This script reads data from BigQuery and writes it to Google Sheets.

Purpose: - query the BigQuery table - write results to a CSV file - open or create the target Google Sheet - clear old data - write the new rows into the sheet

Why it matters: - this is the final output stage of the pipeline - it proves the BigQuery data can be exported to a business-facing tool

5. `scripts/google_auth_utils.py`

This is the credential helper.

Purpose: - make authentication work both locally and in Composer

Credential resolution order: 1. SERVICE_ACCOUNT_JSON 2. GOOGLE_APPLICATION_CREDENTIALS 3. local credentials/service_account.json 4. Secret Manager using GCP_PROJECT_ID and GOOGLE_SECRET_NAME 5. Application Default Credentials fallback

Why it matters: - same scripts can run in local dev and cloud runtime - avoids hardcoding secrets in code

6. `sql/load_sample_bq_table.sql`

This is the SQL template used to create or replace the BigQuery table.

Purpose: - define the sample dataset rows - create the table from an inline UNNEST([...]) structure

Why it matters: - SQL stays separate from Python - the script can reuse the SQL file and inject project/dataset/table names

7. `cloudbuild.yaml`

This is the deployment file used by Cloud Build.

Purpose: - sync the repo contents into the Composer DAG bucket - keep Composer updated whenever GitHub changes are pushed

Sync targets: - learning_airflow/dags -> Composer dags/ - learning_airflow/scripts -> Composer dags/scripts/ - learning_airflow/sql -> Composer dags/sql/

Why it matters: - this is the CI/CD part of the project - Cloud Build moves code from GitHub to Composer automatically

End-to-End Runtime Flow

Step 1: Developer updates code

A change is made in GitHub, for example: - modify SQL rows - update DAG task order - change script logic

Step 2: Cloud Build deploys the change

Cloud Build syncs the changed files to the Composer bucket.

Step 3: Composer picks up the DAG

Composer scans the DAG folder and loads the updated DAG code.

Step 4: Airflow runs the DAG

The monday DAG runs: - run_sql_first - load_sheet_second

Step 5: BigQuery table is refreshed

The SQL creates or replaces the table with the sample rows.

Step 6: Google Sheet is updated

The export script reads from BigQuery and updates the sheet with the newest table data.

Important Debugging Lessons

1. Wrong deployment path

- Issue: DAG did not appear in Composer.
- Fix: corrected Cloud Build sync paths in `cloudbuild.yaml`.

2. Old DAGs still visible

- Issue: deleted DAGs kept showing in UI.
- Fix: used delete-sync (`rsync -d`) so stale files are removed from bucket.

3. Permission error in BigQuery

- Issue: service account authenticated but query failed with 403.
- Fix: granted required BigQuery IAM permissions (like job creation).

4. SQL update not reflected in Sheets

- Issue: BigQuery table was not refreshed due to SQL/deployment mismatch.
- Fix: corrected SQL syntax, redeployed SQL, reran DAG.

5. Local vs cloud credential mismatch

- Issue: cloud workers cannot rely on local credential file paths.
- Fix: used credential fallback strategy (env/file/Secret Manager/ADC).

More detail (where and how):

- The fallback logic is implemented in `learning_airflow/scripts/google_auth_utils.py`.
- Main function used by runtime scripts: `load_google_credentials(scopes)`.
- That function first calls `load_service_account_info()` and only if that fails, it falls back to `google.auth.default(...)` (Application Default Credentials).

Exact fallback order configured:

1. `SERVICE_ACCOUNT_JSON` (raw JSON in env var)
2. `GOOGLE_APPLICATION_CREDENTIALS` (path to key file)
3. `credentials/service_account.json` (local default file)
4. Secret Manager (`GCP_PROJECT_ID + GOOGLE_SECRET_NAME / SERVICE_ACCOUNT_SECRET_NAME`)
5. ADC via `google.auth.default(scopes=scopes)`

Where it is consumed:

- `learning_airflow/scripts/load_sample_data_to_bq.py`
- `learning_airflow/scripts/load_bq_to_gsheet.py`

Why this solved cloud failures:

- Local runs can use file-based credentials.
- Composer workers can use ADC (workload identity/service account) when local key files are not present.

6. Local works, Composer fails

- Issue: dependency/runtime differences between local and Composer.
- Fix: installed required PyPI packages in Composer and verified deployed script versions.

7. Best debugging approach used

- Started from Airflow task logs.
- Verified deployed code in Composer bucket.
- Checked IAM/API/dependencies.
- Retested after each fix, one change at a time.

Interview Talking Points

You can describe the project like this:

I built a GitHub-driven Airflow CI/CD pipeline on Cloud Composer. Cloud Build deploys DAGs, scripts, and SQL to the Composer bucket on push. The monday DAG orchestrates two Python tasks: first it loads a BigQuery table from SQL,

then it reads the table and writes the data into Google Sheets. I designed the auth layer to work locally and in cloud using service account JSON, Secret Manager, and ADC fallback.

Good Follow-Up Topics For Interviewers

Be ready to explain: - why Cloud Composer was used instead of self-managed Airflow - why Cloud Build was chosen for deployment - how the DAG picks up the latest code - how the scripts are separated from the DAG - how credentials are handled safely - how you debugged deployment and permission failures

Simple answer examples:

- **Why Cloud Composer instead of self-managed Airflow?**
 - Composer is managed by Google, so I do not maintain servers, scheduler, or upgrades.
 - It let me focus on DAG logic and data workflow instead of infrastructure.
- **Why Cloud Build for deployment?**
 - Cloud Build runs automatically on GitHub push.
 - It gives a simple CI/CD path: push code -> deploy to Composer bucket.
- **How does DAG pick up latest code?**
 - Cloud Build syncs DAG, scripts, and SQL files into Composer DAG bucket.
 - Composer scheduler scans that folder and loads updates automatically.
- **Why separate scripts from DAG?**
 - DAG handles orchestration only (task order and schedule).
 - Scripts handle business logic (BigQuery load and Sheets export).
- **How are credentials handled safely?**
 - I used a fallback strategy in one auth utility file.
 - It supports env vars, Secret Manager, and ADC, so no hardcoded secrets.
 - **ADC means Application Default Credentials.** It is Google Cloud's built-in way for apps to automatically use the runtime service account identity.
 - In this project, fallback logic is in `scripts/google_auth_utils.py`.
 - The helper first tries explicit sources:
 - `SERVICE_ACCOUNT_JSON` env var
 - `GOOGLE_APPLICATION_CREDENTIALS` file path
 - `local_credentials/service_account.json`
 - Secret Manager (`GCP_PROJECT_ID + GOOGLE_SECRET_NAME`)
 - If none of those exist, it automatically falls back to `google.auth.default(...)` (ADC).
 - In Composer, this ADC fallback uses the Composer worker service account, so tasks can run without local key files.
 - The DAG scripts that use this helper are:
 - `scripts/load_sample_data_to_bq.py`
 - `scripts/load_bq_to_gsheet.py`
 - Security benefit: credentials are not hardcoded in DAG/script logic, and cloud runtime can use identity-based auth.
- **How did you debug failures?**
 - I started from Airflow task logs.
 - Then I checked deployed files, IAM permissions, and dependencies one by one.

Files to Mention in Interview

- learning_airflow/dags/monday.py
- learning_airflow/dags/util/common_utils.py
- learning_airflow/scripts/load_sample_data_to_bq.py
- learning_airflow/scripts/load_bq_to_gsheet.py
- learning_airflow/scripts/google_auth_utils.py
- learning_airflow/sql/load_sample_bq_table.sql
- learning_airflow/cloudbuild.yaml

Short Summary

This project shows a complete production-style pattern:

- GitHub for source control
- Cloud Build for CI/CD
- Cloud Composer for orchestration
- BigQuery as the data warehouse
- Google Sheets as the final output
- Secret Manager / ADC for secure authentication